Linux aufräumen

Stellen die unbemerkt Platz fressen

Ubuntu und andere Linux-Distributionen räumen nicht vollständig hinter sich auf. Über Monate sammeln sich Gigabytes an Daten an, die niemand mehr braucht: alte Snap-Revisionen, der Snap-Download-Cache, APT-Paketcaches, ausgediente Kernel, aufgeblähte Logs. Dieses Dokument zeigt, wo der Platz verschwindet und wie man ihn zurückholt.

Getestet auf Ubuntu 24.04. Die meisten Schritte gelten für jede Debian-basierte Distribution. Stand: Oktober 2026.

Schnellcheck: Wo stehen wir?

```
df -h /
```

Ab 85 % wird es Zeit zum Aufräumen. Ab 95 % streikt `apt upgrade` und die Log-Rotation schlägt fehl.

Die üblichen Verdächtigen auf einen Blick:

```
sudo du -sh /var/lib/snapd/snaps/ /var/lib/snapd/cache/ \
  /var/cache/apt/archives/ /var/log/journal/ /boot/ \
  ~/.cache/thumbnails/ ~/.local/share/Trash/ 2>/dev/null
```

> Die Werte für `snaps/` und `cache/` überschneiden sich teilweise, siehe Abschnitt 1.

1. Snap: alte Revisionen und Download-Cache

Snap behält auf Ubuntu standardmäßig die aktuelle und die vorherige Revision jedes Snaps, damit man mit `snap revert` zurückkehren kann. Bei großen Anwendungen wie Firefox, Thunderbird oder LibreOffice und den Laufzeitumgebungen (`gnome-*`, `core*`) belegt das schnell mehrere GiB doppelt.

Alte Revisionen entfernen

Deaktivierte Revisionen bleiben liegen, bis snapd sie bei einer späteren Aktualisierung verdrängt. Vorher prüfen, was entfernt wird:

```
LANG=C snap list --all | awk '/disabled/{print $1, $3}'
```

Entfernen:

```
LANG=C snap list --all | awk '/disabled/{print $1, $3}' |
  while read -r name rev; do
    sudo snap remove "$name" --revision="$rev"
  done
```

> `LANG=C` sorgt dafür, dass die Ausgabe unabhängig von der Systemsprache gleich aussieht. Die laufende Revision bleibt unangetastet. `snap revert` ist danach erst nach der nächsten Aktualisierung wieder möglich.

Download-Cache leeren

snapd legt heruntergeladene Snaps zusätzlich unter `/var/lib/snapd/cache/` ab. Die Dateien dort tragen keine Endung `.snap`, sondern eine lange Prüfsumme als Namen, zum Beispiel `03a76f51...bdf691d64`. Ein Befehl wie `rm /var/lib/snapd/cache/*.snap` findet deshalb nichts.

Die Dateien sind Hardlinks auf die installierten Snaps. Solange eine Revision installiert ist, belegen beide Einträge den Platz nur einmal. Wird die Revision entfernt, bleibt die Datei im Cache als einziger Verweis übrig und belegt den Platz weiter.

Wie viel davon verwaist ist:

```
sudo find /var/lib/snapd/cache -type f -links 1 -printf '%s\n' |
  awk '{s+=$1} END {printf "%.1f GiB verwaist\n", s/2^30}'
```

Leeren:

```
sudo find /var/lib/snapd/cache -type f -delete
```

> Gefahrlos: Noch installierte Snaps behalten ihre Daten über `/var/lib/snapd/snaps/`, nur die verwaisten Kopien werden freigegeben.

Zukünftige Revisionen begrenzen

Die Einstellung `refresh.retain` legt fest, wie viele Revisionen pro Snap aufbewahrt werden. Erlaubt sind 2 bis 20. Standard ist 2 auf klassischem Ubuntu und 3 auf Ubuntu Core.

```
sudo snap get system refresh.retain
sudo snap set system refresh.retain=2
```

> Eine leere Ausgabe bei `get` bedeutet: Standardwert aktiv. Das explizite Setzen lohnt sich, falls der Wert irgendwann erhöht wurde.

2. APT-Paketcache

APT speichert jede heruntergeladene `.deb`-Datei in `/var/cache/apt/archives/` und lässt sie dort liegen.

Cache-Größe anzeigen:

```
sudo du -sh /var/cache/apt/archives/
```

Nur Pakete entfernen, die in den Paketquellen nicht mehr verfügbar sind:

```
sudo apt autoclean
```

Komplett leeren:

```
sudo apt clean
```

> `autoclean` ist die konservativere Variante. `clean` holt mehr Platz, benötigte Pakete werden beim nächsten `apt install` einfach neu heruntergeladen.

3. Alte Kernel

Jedes Kernel-Update installiert einen neuen Kernel neben die bisherigen, unter `/boot/` und mit Modulen unter `/lib/modules/`. Jeder Kernel belegt ca. 300 bis 500 MiB. Ubuntu markiert alte Kernel als entfernbar, entfernt sie aber erst, wenn `apt autoremove` läuft.

Laufenden Kernel und installierte Kernel anzeigen:

```
uname -r
dpkg -l 'linux-image-*' | grep '^ii'
```

Entfernen:

```
sudo apt autoremove --purge
```

> Ubuntu behält dabei den laufenden, den neuesten und einen weiteren Kernel als Rückfallebene (`/etc/apt/apt.conf.d/01autoremove-kernels`). `autoremove` entfernt aber auch alle anderen automatisch installierten Pakete, die nichts mehr braucht. Die Liste vor dem Bestätigen lesen. Ein Paket, das bleiben soll, mit `sudo apt-mark manual <paket>` schützen.

Wenn `autoremove` einen alten Kernel nicht erfasst (kann bei manuell installierten Kerneln passieren), gezielt entfernen, aber nie den laufenden:

```
# Beispiel: konkreten alten Kernel entfernen
sudo apt purge linux-image-6.8.0-31-generic \
  linux-headers-6.8.0-31-generic \
  linux-modules-6.8.0-31-generic
```

> Den GRUB-Eintrag aktualisiert apt dabei selbst, ein `update-grub` von Hand ist nicht nötig.

4. Journal-Logs

systemd-journald speichert Logs in `/var/log/journal/`. Ohne eigene Einstellung darf das Journal bis zu 10 % des Dateisystems belegen, höchstens 4 GiB.

Aktuellen Platzbedarf anzeigen:

```
journalctl --disk-usage
```

Auf eine Größe oder ein Alter kürzen:

```
sudo journalctl --vacuum-size=200M
sudo journalctl --vacuum-time=7d
```

Dauerhaftes Limit setzen

Als eigene Datei unter `journald.conf.d/`, damit die Einstellung Paket-Updates übersteht:

```
sudo mkdir -p /etc/systemd/journald.conf.d
printf '[Journal]\nSystemMaxUse=200M\n' |
  sudo tee /etc/systemd/journald.conf.d/groesse.conf
sudo systemctl restart systemd-journald
```

> `SystemMaxUse` begrenzt den gesamten Platz des Journals auf der Festplatte. journald löscht danach selbstständig die ältesten Einträge.

5. Thumbnails

Dateimanager und Bildbetrachter legen Vorschaubilder in `~/.cache/thumbnails/` an, für jedes Bild, PDF und Video, das jemals angezeigt wurde. Auch für längst gelöschte Dateien.

Platzbedarf:

```
du -sh ~/.cache/thumbnails/
```

Komplett leeren (werden bei Bedarf neu erzeugt):

```
rm -rf ~/.cache/thumbnails/*
```

Grenzen unter GNOME

GNOME löscht Vorschaubilder selbst, die 180 Tage nicht benutzt wurden, und begrenzt den Cache auf 512 MiB. Engere Grenzen pro Benutzer:

```
gsettings set org.gnome.desktop.thumbnail-cache maximum-age 30
gsettings set org.gnome.desktop.thumbnail-cache maximum-size 256
```

> Alter in Tagen, Größe in MiB. Der Wert `-1` schaltet die jeweilige Grenze ab.

6. Papierkorb

Gelöschte Dateien landen unter `~/.local/share/Trash/` und belegen weiter Platz. Auf anderen Laufwerken liegt ein eigener Papierkorb im Ordner `.Trash-1000/` (Benutzernummer) im Wurzelverzeichnis des Laufwerks.

Platzbedarf:

```
du -sh ~/.local/share/Trash/ 2>/dev/null
```

Leeren, auf allen eingebundenen Laufwerken:

```
gio trash --empty
```

Automatisch leeren unter GNOME

GNOME kann Dateien, die länger als eine bestimmte Zeit im Papierkorb liegen, selbst löschen. Maßgeblich ist dabei das Löschdatum, nicht das Alter der Datei. Ab Werk ist das ausgeschaltet.

Einstellungen → Datenschutz und Sicherheit → Dateichronik und Papierkorb → „Automatisch den Papierkorb leeren“ einschalten. Oder per Befehl:

```
gsettings set org.gnome.desktop.privacy remove-old-trash-files true
gsettings set org.gnome.desktop.privacy old-files-age 30
```

Automatisches Cleanup per systemd-Timer

Die Systembereiche (Snap, APT, Kernel, Journal) brauchen root-Rechte und lassen sich per Skript und systemd-Timer regelmäßig erledigen. Thumbnails und Papierkorb gehören dem jeweiligen Benutzer: Dafür sind die GNOME-Einstellungen aus Abschnitt 5 und 6 der saubere Weg.

> Wer die frühere Fassung dieser Anleitung eingerichtet hat: Skript- und Unit-Namen sind gleich geblieben. Skript ersetzen, Service-Datei ersetzen, `daemon-reload`, fertig. Der Snap-Cache wurde mit der alten Fassung nicht geleert.

Skript: /usr/local/sbin/linux-cleanup.sh

```
#!/usr/bin/env bash
# linux-cleanup.sh: entfernt Snap-, APT- und Journal-Altlasten
# Ohne Optionen ruhig: Fortschrittspunkte, Ergebnis, Fehler.

set -uo pipefail

JOURNAL_MAX="200M"
PROBE=0; KERNEL=0; DEBUG=0

hilfe() {
  cat <<'EOF'
Aufruf: linux-cleanup.sh [-n] [-k] [-d] [-h]
  -n  Probelauf: zeigt nur, was ausgeführt würde
  -k  zusätzlich apt autoremove --purge (u. a. alte Kernel)
  -d  Debug: alle Befehle und Ausgaben anzeigen
  -h  diese Hilfe
EOF
}

while getopts "nkdh" opt; do
  case "$opt" in
    n) PROBE=1 ;;
    k) KERNEL=1 ;;
    d) DEBUG=1 ;;
    h) hilfe; exit 0 ;;
```

```
    *) hilfe; exit 2 ;;
  esac
done

if [[ $EUID -ne 0 ]]; then
  echo "Bitte mit sudo bzw. als root ausführen." >&2
  exit 1
fi

FEHLER=0

# $1 = Beschreibung, Rest = Befehl
schritt() {
  local text="$1"; shift
  if (( PROBE )); then
    printf '[Probelauf] %s: %s\n' "$text" "$*"; return 0
  fi
  if (( DEBUG )); then
    echo "--- $text: $*"
    "$@" || { echo "FEHLER: $text" >&2; FEHLER=$((FEHLER+1)); }
  else
    local out
    if ! out=$("$@" 2>&1); then
      printf '\nFEHLER: %s\n%s\n' "$text" "$out" >&2
      FEHLER=$((FEHLER+1))
    fi
    printf '.'
  fi
}

belegt() { df --output=used -B1 / | tail -n 1; }
vorher=$(belegt)

# Snap
if command -v snap >/dev/null 2>&1; then
  schritt "Snap: refresh.retain=2" snap set system refresh.retain=2
  while read -r name rev; do
    schritt "Snap: $name Rev. $rev" \
      snap remove "$name" --revision="$rev"
  done < <(LANG=C snap list --all | awk '/disabled/{print $1, $3}')
  schritt "Snap: Download-Cache" \
    find /var/lib/snapd/cache -type f -delete
fi

# APT
schritt "APT: Paketcache" apt-get clean
if (( KERNEL )); then
  schritt "APT: autoremove" apt-get -y autoremove --purge
fi

# Journal
schritt "Journal: auf $JOURNAL_MAX" \
  journalctl --vacuum-size="$JOURNAL_MAX"

# Ergebnis
(( PROBE || DEBUG )) || echo
if (( ! PROBE )); then
  frei=$(( vorher - $(belegt) )); (( frei < 0 )) && frei=0
  echo "Freigegeben: $(numfmt --to=iec-i --suffix=B "$frei")"
  df -h / | awk 'NR==2 {print "Belegung /: " $5 ", frei: " $4}'
fi

if (( FEHLER > 0 )); then
  echo "$FEHLER Schritt(e) fehlgeschlagen." >&2
  exit 1
fi
```

Installieren und einmal von Hand testen:

```
sudo install -m 755 linux-cleanup.sh /usr/local/sbin/linux-cleanup.sh
sudo linux-cleanup.sh -n
sudo linux-cleanup.sh
```

> Ohne Option erscheint pro Schritt ein Punkt, am Ende die freigegebene Menge. `-k` nimmt `apt autoremove --purge` hinzu. Das ist bewusst nicht Standard, weil `autoremove` neben alten Kerneln auch andere Pakete entfernen kann (siehe Abschnitt 3).

systemd-Service

`/etc/systemd/system/linux-cleanup.service`:

```
[Unit]
Description=Linux-Cleanup (Snap, APT, Journal)
After=snapd.service

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/linux-cleanup.sh
Nice=19
IOSchedulingClass=idle
```

> `Nice=19` und `IOSchedulingClass=idle` geben dem Aufräumen die niedrigste Priorität, damit es die Arbeit am Rechner nicht bremst. Soll `-k` mitlaufen: `ExecStart=/usr/local/sbin/linux-cleanup.sh -k`.

systemd-Timer

`/etc/systemd/system/linux-cleanup.timer`:

```
[Unit]
Description=Linux-Cleanup wöchentlich

[Timer]
OnCalendar=Sun 03:00
Persistent=true
RandomizedDelaySec=1800

[Install]
WantedBy=timers.target
```

> `Persistent=true` sorgt dafür, dass ein verpasster Lauf (Rechner war aus) beim nächsten Start nachgeholt wird. `RandomizedDelaySec` verhindert, dass alle Timer gleichzeitig starten.

Aktivieren und prüfen

```
sudo systemctl daemon-reload
sudo systemctl enable --now linux-cleanup.timer
```

Status prüfen:

```
systemctl list-timers linux-cleanup.timer
```

Manuell auslösen und Ergebnis ansehen:

```
sudo systemctl start linux-cleanup.service
journalctl -u linux-cleanup.service -e
```

Zusammenfassung

Stelle	Pfad	Typisch	Befehl
Snap (alte Rev.)	/var/lib/snapd/snaps/	2–15 GiB	snap remove --revision
Snap (Cache)	/var/lib/snapd/cache/	1–10 GiB	find ... -type f -delete
APT-Cache	/var/cache/apt/archives/	0,5–3 GiB	apt clean
Alte Kernel	/boot/, /lib/modules/	0,3–2 GiB	apt autoremove --purge
Journal	/var/log/journal/	0,5–4 GiB	journalctl --vacuum-size=200M
Thumbnails	~/.cache/thumbnails/	0,1–1 GiB	gsettings ... maximum-age
Papierkorb	~/.local/share/Trash/	variabel	gio trash --empty

Lizenz

MIT

Erstellt im Juni 2026 · Clear Consulting LLC · klare-beratung.de